

Dossier Technique

Projet PGI Pro

Progiciel de Gestion Intégré Web

Auteur : HAMADENE Ilies, LACOMBE Alexis

Université : Université Paris Nanterre

Date : 13 novembre 2025

Table des matières

1	Introduction	2
2	Architecture générale	2
2.1	Vue d'ensemble	2
3	Technologies utilisées	2
4	Structure du code source	3
4.1	Fichiers principaux	3
4.2	Structure HTML	3
4.3	Organisation CSS	3
4.4	Organisation JavaScript	3
5	Gestion des données	4
5.1	Structure de stockage	4
5.2	Persistance locale	4
5.3	Session utilisateur	4
6	Fonctionnement global	4
6.1	Cycle d'utilisation	4
6.2	Sécurité et rôles	4
7	Performances et maintenance	5
7.1	Performances	5
7.2	Maintenance	5
8	Perspectives d'évolution	5
	Conclusion	5

1 Introduction

Le projet **PGI Pro** est une application web locale permettant la gestion intégrée d'une PME : stocks, achats, ventes et ressources humaines. Ce dossier technique présente les choix technologiques, la structure du code, l'architecture interne et le fonctionnement global du système.

2 Architecture générale

2.1 Vue d'ensemble

L'application repose sur une architecture entièrement **front-end**, sans serveur :

- **Frontend** : HTML, CSS, JavaScript (ES6) ;
- **Stockage local** : `localStorage` et `sessionStorage` du navigateur ;
- **Exécution** : via navigateur web (mode local, sans déploiement serveur).

3 Technologies utilisées

- **HTML5** : structure des pages et formulaires ;
- **CSS3** : mise en page responsive et design moderne ;
- **JavaScript (ES6)** : gestion de la logique applicative et du DOM ;
- **Web Storage API** :
 - `localStorage` : persistance des données ;
 - `sessionStorage` : gestion de session.
- **Aucun framework externe** — le code est 100% natif.

4 Structure du code source

4.1 Fichiers principaux

Le projet se compose d'un seul fichier principal :

pgi.html

HTML : structure de l'application
CSS : style complet intégré dans <style>
JS : logique applicative via <script>

4.2 Structure HTML

L'interface est organisée en deux parties :

- **Page de connexion** — formulaire d'authentification ;
- **Application principale** — affichée après connexion :
 - Barre latérale de navigation (dynamique selon le rôle) ;
 - Zone de contenu principale (modules et tableau de bord).

4.3 Organisation CSS

Le style repose sur une charte cohérente :

- Variables globales (`:root`) pour les couleurs ;
- Grilles CSS (`display: grid`) pour la disposition ;
- Classes modulaires : `.login-page`, `.module-card`, `.stat-card`, etc. ;
- Transitions et ombres pour un rendu fluide et professionnel.

4.4 Organisation JavaScript

La logique applicative est contenue dans le bloc <script> :

- **Connexion utilisateur** : `handleLogin()` ;
- **Initialisation** : `initializeApp()` ;
- **Modules métiers** :
 - `submitStock()`, `renderStocks()`, `deleteStock()` ;
 - `submitAchat()`, `renderAchats()`, `deleteAchat()` ;
 - `submitVente()`, `renderVentes()`, `deleteVente()` ;
 - `submitRH()`, `renderRH()`, `deleteRH()`.
- **Persistance des données** : `saveData()` via `localStorage.setItem()` ;
- **Navigation** : fonction `showPage(pageId)`.

5 Gestion des données

5.1 Structure de stockage

Chaque entité est sauvegardée sous forme JSON :

```
— stocks = [{id, ref, nom, qte, prix}];  
— achats = [{id, date, four, prodNom, qte, total}];  
— ventes = [{id, date, client, prodNom, qte, total, vendeur}];  
— salaries = [{id, nom, prenom, poste, salaire}].
```

5.2 Persistance locale

Sauvegarde et restauration automatique :

```
localStorage.setItem('stocks', JSON.stringify(stocks));  
let stocks = JSON.parse(localStorage.getItem('stocks'))  
|| [];
```

5.3 Session utilisateur

Gérée avec `sessionStorage` :

```
sessionStorage.setItem('currentUser',  
                        JSON.stringify(currentUser));
```

La session est supprimée à la déconnexion.

6 Fonctionnement global

6.1 Cycle d'utilisation

1. L'utilisateur se connecte avec ses identifiants ;
2. Le système charge les données locales ;
3. Les modules s'affichent selon le rôle (admin/employé) ;
4. Les actions (ajout, suppression, modification) modifient le stockage local ;
5. L'utilisateur peut se déconnecter à tout moment.

6.2 Sécurité et rôles

- Vérification du rôle avant toute action critique ;
- Interface adaptée : menus et pages selon profil ;
- Aucune donnée sensible transmise en réseau.

7 Performances et maintenance

7.1 Performances

- Exécution instantanée (aucune requête réseau) ;
- Chargement rapide et fluide ;
- Faible consommation mémoire (petits jeux de données).

7.2 Maintenance

- Code unique, facile à éditer ;
- Séparation logique claire entre HTML, CSS et JS ;
- Possibilité d'externaliser les modules JS/CSS pour une future extension.

8 Perspectives d'évolution

- Connexion à une base de données externe (SQLite, MySQL) ;
- Ajout de rapports statistiques et graphiques interactifs ;
- Gestion des fournisseurs et clients complète ;
- Génération automatique de factures PDF.

Conclusion

Le dossier technique du projet **PGI Pro** décrit son architecture, sa logique interne et son fonctionnement. L'approche 100% front-end rend le système simple, portable et parfaitement adapté à un usage académique ou PME.